README

Lab 2 Exercise 2: Gesture Classification

Real-Time Gesture Classification Using Processing, Webcam Input, Pixel Feature Extraction,

OSC Communication, Wekinator, and KNN

`GestureInput.pde` + `GestureOutput.pde`

Contents

1 Purpose of This README **4**

2 Objective of Exercise 2 **4**

3 Main Concept of the Exercise **4**

4 Required Tools and Libraries **5**

4.1 Processing 5

4.2 Processing Video Library 5

4.3 oscP5 Library 5

4.4 netP5 Library 5

4.5 Wekinator 5

5 Deliverables for the Exercise **5**

6 Complete System Architecture **6**

7 Current Tested Configuration **6**

8 The Input Sketch: `GestureInput.pde` **7**

8.1 Role of the Input Sketch 7

8.2 Main Tasks of the Input Sketch 7

9 Important Input Sketch Variables **7**

10 Webcam Capture **8**

11 Image Cropping **8**

12 Downsampling **8**

13 Grayscale Feature Extraction **9**

14 RGB Feature Extraction Option **9**

15 OSC Sending from Input Sketch to Wekinator **10**

16 Input Sketch Debugging Interface **10**

17 Wekinator Project Configuration **10**

17.1 Input Configuration 10

17.2 Output Configuration 11

17.3 Classifier Configuration 11

18 Training Data in the Wekinator Project **11**

19 What KNN Means **11**

20 Why k = 3 Is Used **12**

21 Training Procedure **12**

21.1 Step 1 — Run the Input Sketch . . . 12
21.2 Step 2 — Open Wekinator . . . 12
21.3 Step 3 — Start Listening . . . 12
21.4 Step 4 — Record Class 1 . . . 13
21.5 Step 5 — Record Class 2 . . . 13
21.6 Step 6 — Train the Model . . . 13
21.7 Step 7 — Run the Output Sketch . . . 13
21.8 Step 8 — Run Wekinator . . . 13
21.9 Step 9 — Test the System . . . 13

22 The Output Sketch: `GestureOutput.pde` **13**
22.1 Role of the Output Sketch . . . 13
22.2 Main Tasks of the Output Sketch . . . 14

23 Important Output Sketch Variables **14**

24 Receiving OSC Output from Wekinator **14**

25 Class-to-Gesture Mapping **15**

26 Output Visualization **15**

27 Correct Running Order **15**
27.1 During Training . . . 15
27.2 During Testing and Demonstration . . . 16

28 Important Practical Training Notes **16**
28.1 The Hand Must Be Inside the Crop Box . . . 16
28.2 Keep the Background Simple . . . 16
28.3 Keep Lighting Stable . . . 16
28.4 Record Slight Variations . . . 16
28.5 Keep Classes Balanced . . . 16
28.6 Retrain After Changes . . . 17

29 Discussion: Image Resolution **17**
29.1 200×200 Resolution . . . 17
29.2 100×100 Resolution . . . 17
29.3 50×50 Resolution . . . 17

30 Discussion: Grayscale vs RGB **17**
30.1 Grayscale . . . 17
30.2 RGB . . . 18
30.3 Current Choice . . . 18

31 Feature Configuration Table **18**

32 How the Current Project Meets the Exercise Requirements **18**

33 Results **19**

34 Common Problems and Debugging **19**
34.1 Wekinator Does Not Receive Inputs . . . 19
34.2 Wekinator Receives Input but Training Is Bad . . . 20
34.3 Output Sketch Does Not Change . . . 20
34.4 Output Shows Wrong Gesture . . . 20

35 Important Notes About the Green Crop Box **20**

36 What Was Learned **21**
36.1 Real-Time Video Processing . . . 21
36.2 Image Preprocessing . . . 21
36.3 Feature Extraction . . . 21
36.4 Dimensionality . . . 21
36.5 OSC Communication . . . 21
36.6 Supervised Learning . . . 21
36.7 KNN Classification . . . 21
36.8 Interactive Machine Learning . . . 21

37 Possible Improvements **21**
37.1 Movable Crop Box . . . 21
37.2 Larger Feature Size . . . 21
37.3 Background Removal . . . 21
37.4 More Gesture Classes . . . 21
37.5 More Robust Training . . . 22

38 Final Project Summary **22**

39 Final Conclusion **22**

1. Purpose of This README

This README is the complete report for **Lab 2 — Exercise 2: Gesture Classification**.

It explains the whole project from beginning to end:

1. The objective of the exercise.
2. The required tools and libraries.
3. The complete system architecture.
4. The Processing input sketch.
5. The Wekinator project configuration.
6. The KNN classifier and the training process.
7. The Processing output sketch.
8. The correct running order.
9. The final tested configuration.
10. The results and observations.
11. The discussion about image resolution, grayscale, RGB, dimensionality, and classification performance.
12. Common problems and debugging advice.
13. How the project satisfies the requirements of the exercise.

The goal is to make the whole exercise understandable as a complete interactive machine learning system.

2. Objective of Exercise 2

The objective of Exercise 2 is to build a **real-time gesture classification system**.

The system must use a webcam to capture a live video image, extract meaningful visual features from that image, send those features to Wekinator through OSC, train a classifier, and display the recognized gesture in real time.

In simple words, the system should do this:

```
User shows a gesture to the webcam
        ↓
Processing captures the image
        ↓
Processing crops and downsamples the image
        ↓
Processing converts pixels into numerical features
        ↓
Processing sends those features to Wekinator
        ↓
Wekinator classifies the gesture
        ↓
Wekinator sends the predicted class to Processing
        ↓
Processing displays the recognized gesture label
```

The exercise is about understanding how a visual gesture can be represented as numbers and classified by a machine learning algorithm.

3. Main Concept of the Exercise

A computer does not directly understand a hand gesture like a human does.

A human can look at an image and say:

```
This is an open hand.
```

But Wekinator receives only numbers.

Therefore, the Processing input sketch must transform the camera image into a **feature vector**.

In this project, the feature vector is created from a downsampled grayscale image:

```
50 × 50 grayscale pixels = 2500 numerical values
```

Each value represents the brightness of one pixel.

For example:

```
0.12, 0.15, 0.18, 0.20, ..., 0.76
```

Wekinator learns the relationship between these pixel patterns and gesture classes.

For the current project:

```
Class 1 → Open Hand
Class 2 → Closed Fist
```

4. Required Tools and Libraries

4.1. Processing

Processing is used to write and run the two sketches:

1. The input sketch: `GestureInput.pde`
2. The output sketch: `GestureOutput.pde`

Processing is responsible for webcam capture, image processing, visual display, and OSC communication.

4.2. Processing Video Library

The Processing Video library is used in the input sketch to access the webcam.

The important class is:

```
Capture cam;
```

This object represents the live camera stream.

4.3. oscP5 Library

The oscP5 library is used for OSC communication.

OSC means **Open Sound Control**. It allows independent programs to exchange real-time messages.

In this exercise, OSC connects:

```
Processing input sketch → Wekinator → Processing output sketch
```

4.4. netP5 Library

The netP5 library works together with oscP5. It is used to define a destination address for OSC messages.

For example:

```
NetAddress wekinator;
```

This tells Processing where Wekinator is listening.

4.5. Wekinator

Wekinator is the machine learning tool used in this exercise.

It receives feature vectors from the input sketch, records examples for each gesture class, trains a classifier, and sends the predicted class to the output sketch.

In this project, Wekinator uses a **K-Nearest Neighbors classifier** with:

```
k = 3
```

5. Deliverables for the Exercise

The exercise requires the following deliverables:

1. A Processing project for data acquisition and feature extraction.

2. A Processing project for visualization of the classification output.
3. A Wekinator project used for training and testing the classifier.
4. A short report discussing the system, the feature configurations, and classification performance.

This project contains:

Deliverable	File / Component
Input Processing sketch	`GestureInput.pde`
Output Processing sketch	`GestureOutput.pde`
Wekinator project	`WekinatorProject.zip`
Report	This README

6. Complete System Architecture

The full system has three main parts:

```
1. Processing input sketch
2. Wekinator classifier
3. Processing output sketch
```

The complete flow is:

```
User gesture
    ↓
Webcam
    ↓
Processing input sketch
    ↓
200×200 crop
    ↓
50×50 grayscale image
    ↓
2500 numerical features
    ↓
OSC message /wek/inputs
    ↓
Wekinator on port 6448
    ↓
KNN classifier
    ↓
Predicted class number
    ↓
OSC message /wek/outputs
    ↓
Processing output sketch on port 12000
    ↓
Displayed gesture label
```

This architecture separates the system into clear responsibilities.

Component	Responsibility
Input Processing sketch	Captures webcam image and extracts features
Wekinator	Learns and predicts gesture classes
Output Processing sketch	Displays the predicted gesture label

7. Current Tested Configuration

The current project uses the following tested configuration:

Item	Value
Crop size	200×200 pixels
Feature image size	50×50 pixels
Color mode	Grayscale
Number of Wekinator inputs	2500

Item	Value
Wekinator outputs	1
Output type	Classification
Number of gesture classes	2
Classifier	K-Nearest Neighbors
KNN value	k = 3
Class 1	Open Hand
Class 2	Closed Fist
Input OSC message	`/wek/inputs`
Input OSC port	6448
Output OSC message	`/wek/outputs`
Output OSC port	12000

The Wekinator project contains a balanced training dataset:

Class	Number of examples
Class 1	637
Class 2	594
Total	1231

This balance is good because neither class strongly dominates the other.

8. The Input Sketch: `GestureInput.pde`

8.1. Role of the Input Sketch

The input sketch is responsible for converting webcam images into numerical features.

It does not classify the gesture by itself. It only prepares the input data for Wekinator.

Its responsibility is:

```
Webcam image → Feature vector → Wekinator
```

8.2. Main Tasks of the Input Sketch

The input sketch performs these steps:

1. Starts the webcam.
2. Displays the camera preview.
3. Extracts a 200×200 crop from the center of the image.
4. Resizes the crop to 50×50 pixels.
5. Converts each pixel to a grayscale brightness value.
6. Normalizes each value from 0–255 to 0–1.
7. Adds all values to an OSC message.
8. Sends the message to Wekinator at `/wek/inputs` on port 6448.
9. Shows preview panels for debugging.
10. Allows the user to turn sending on or off with the spacebar.

9. Important Input Sketch Variables

The most important variables in the input sketch are:

```
String wekHost = "127.0.0.1";
int wekPort = 6448;
String oscAddress = "/wek/inputs";
```

These variables define where the input sketch sends OSC data.

```
int cropSize = 200;
int featureSize = 50;
boolean useRGB = false;
```

These variables define the feature extraction configuration.

They mean:

```
Crop a 200×200 region
Resize it to 50×50
Use grayscale instead of RGB
```

The resulting number of inputs is:

```
50 × 50 = 2500 inputs
```

10. Webcam Capture

The input sketch uses:

```
Capture cam;
```

This object stores the live camera image.

The camera is started in `setup()`:

```
cam = new Capture(this, cameras[0]);
cam.start();
```

The camera frame is updated using:

```
void captureEvent(Capture c) {
  c.read();
}
```

This means whenever a new camera frame is ready, Processing reads it.

11. Image Cropping

The exercise requires the central region of the image to be cropped using a square of 200×200 pixels.

The input sketch does this with:

```
cropImg = getCenterCrop(cam, cropSize);
```

The function calculates the starting x and y positions:

```
int startX = (source.width - size) / 2;
int startY = (source.height - size) / 2;
```

Then it extracts the square crop:

```
return source.get(startX, startY, size, size);
```

This crop is important because Wekinator should receive the part of the image that contains the gesture.

Important practical note:

```
The hand must be inside the green crop box during training and testing.
```

If the crop contains mostly the face or background, the classifier will learn the wrong information.

12. Downsampling

After the 200×200 crop is created, the sketch downsamples it:

```
featureImg = cropImg.copy();
featureImg.resize(featureSize, featureSize);
```

With the current value:

```
int featureSize = 50;
```

this creates a 50×50 image.

Downsampling is important because the full 200×200 grayscale crop would create:

```
200 × 200 = 40,000 inputs
```

That is much heavier than:

```
50 × 50 = 2500 inputs
```

The 50×50 image preserves the general shape of the hand while reducing the amount of data sent to Wekinator.

13. Grayscale Feature Extraction

The current project uses grayscale features:

```
boolean useRGB = false;
```

For each pixel, the sketch calculates brightness:

```
float gray = brightness(c) / 255.0;
msg.add(gray);
```

This means each pixel becomes one value between 0 and 1.

The meaning is:

Value	Meaning
0.0	Black / very dark
0.5	Medium gray
1.0	White / very bright

For a 50×50 image, there are 2500 pixels, so the sketch sends 2500 grayscale values.

14. RGB Feature Extraction Option

The sketch also supports RGB mode.

If this variable is changed:

```
boolean useRGB = true;
```

then each pixel sends three values:

```
msg.add(red(c) / 255.0);
msg.add(green(c) / 255.0);
msg.add(blue(c) / 255.0);
```

In RGB mode, the number of inputs is:

```
width × height × 3
```

For 50×50 RGB:

```
50 × 50 × 3 = 7500 inputs
```

RGB keeps color information, but it is heavier and more sensitive to lighting changes.

For open hand vs closed fist, grayscale is usually enough because the difference is mainly in shape, not color.

15. OSC Sending from Input Sketch to Wekinator

The input sketch sends features using OSC.

The OSC message is created with:

```
OscMessage msg = new OscMessage(oscAddress);
```

Since:

```
String oscAddress = "/wek/inputs";
```

the message address is:

```
/wek/inputs
```

All 2500 feature values are added to the message.

Then the message is sent:

```
oscP5.send(msg, wekinator);
```

The destination is:

```
127.0.0.1:6448
```

This must match the Wekinator input settings exactly.

16. Input Sketch Debugging Interface

The input sketch displays helpful debugging information:

1. Full webcam preview.
2. Green crop rectangle.
3. 200×200 crop preview.
4. 50×50 feature image preview.
5. OSC address.
6. Wekinator destination port.
7. Feature size.
8. Color mode.
9. Number of Wekinator inputs.
10. Sending ON/OFF status.

This is very important because the user can visually check whether the hand is actually inside the crop.

The spacebar can pause and resume sending:

```
if (key == ' ') {
  sendToWekinator = !sendToWekinator;
}
```

This helps during setup and debugging.

17. Wekinator Project Configuration

The Wekinator project is configured to receive 2500 inputs and output one classification value.

17.1. Input Configuration

Setting	Value
Receive port	6448
OSC input message	`/wek/inputs`
Number of inputs	2500

This matches the input sketch exactly.

17.2. Output Configuration

Setting	Value
Output OSC message	`/wek/outputs`
Output host	`localhost`
Output port	12000
Number of outputs	1
Output type	Classification
Number of classes	2

This matches the output sketch exactly.

17.3. Classifier Configuration

The project uses:

```
K-Nearest Neighbors classifier
k = 3
```

This means Wekinator classifies a new input by comparing it to the three nearest stored training examples.

18. Training Data in the Wekinator Project

The current Wekinator project contains:

```
Total training examples: 1231
Class 1 examples: 637
Class 2 examples: 594
```

This is a good balance.

Balanced training data is important because if one class has many more examples than the other, the classifier may become biased.

For example, this would be bad:

```
Class 1: 1000 examples
Class 2: 100 examples
```

The current project is much better:

```
Class 1: 637 examples
Class 2: 594 examples
```

19. What KNN Means

KNN means **K-Nearest Neighbors**.

It is a simple machine learning method for classification.

The idea is:

```
A new input arrives
        ↓
Compare it with all stored training examples
        ↓
Find the K closest examples
        ↓
Use majority vote
        ↓
Predict the class
```

In this project:

```
K = 3
```

So Wekinator looks at the three nearest examples.

Example:

```
Nearest example 1 → Class 1
Nearest example 2 → Class 1
Nearest example 3 → Class 2
```

The vote is:

```
Class 1: 2 votes
Class 2: 1 vote
```

Final prediction:

```
Class 1
```

20. Why k = 3 Is Used

Using `k = 3` is more stable than using `k = 1`.

If `k = 1`, Wekinator uses only the single closest example. This can be sensitive to noise or one bad recording.

With `k = 3`, Wekinator uses a small majority vote.

This makes the classification less sensitive to one unusual or noisy example.

For this project, `k = 3` is appropriate because:

1. The dataset has enough examples.
2. The classes are balanced.
3. The gestures may vary slightly between recordings.
4. Majority voting improves stability.

21. Training Procedure

21.1. Step 1 — Run the Input Sketch

Open and run:

```
GestureInput.pde
```

Check that the console shows:

```
Number of Wekinator inputs: 2500
OSC address: /wek/inputs
Sending to: 127.0.0.1:6448
```

Also check that the webcam appears and the green crop box is visible.

21.2. Step 2 — Open Wekinator

Open the Wekinator project or create a new one with these settings:

```
Input port: 6448
Input OSC message: /wek/inputs
Number of inputs: 2500
Output message: /wek/outputs
Output host: localhost
Output port: 12000
Number of outputs: 1
Output type: Classification
Number of classes: 2
```

21.3. Step 3 — Start Listening

Click **Start Listening** in Wekinator.

Wekinator should start receiving values from the input sketch.

21.4. Step 4 — Record Class 1

Select Class 1.

Place the open hand gesture clearly inside the green crop box.

Click **Start Recording**.

Move the hand slightly while keeping the same gesture.

Click **Stop Recording**.

21.5. Step 5 — Record Class 2

Select Class 2.

Place the closed fist gesture clearly inside the same crop box.

Click **Start Recording**.

Move the hand slightly while keeping the same gesture.

Click **Stop Recording**.

21.6. Step 6 — Train the Model

Set the classifier to KNN with:

```
k = 3
```

Click **Train**.

21.7. Step 7 — Run the Output Sketch

Open and run:

```
GestureOutput.pde
```

It listens on:

```
Port: 12000
Message: /wek/outputs
```

21.8. Step 8 — Run Wekinator

Click **Run** in Wekinator.

Do not click **Start Recording** during testing unless you want to add more examples.

21.9. Step 9 — Test the System

Show an open hand.

Expected output:

```
Class 1 → Open Hand
```

Show a closed fist.

Expected output:

```
Class 2 → Closed Fist
```

22. The Output Sketch: `GestureOutput.pde`

22.1. Role of the Output Sketch

The output sketch receives Wekinator's predicted class and displays it as a readable gesture label.

Its responsibility is:

```
Wekinator prediction → Visual feedback
```

It does not use the webcam and does not extract features.

22.2. Main Tasks of the Output Sketch

The output sketch performs these tasks:

1. Opens OSC port 12000.
2. Waits for messages from Wekinator.
3. Accepts only messages with address `/wek/outputs`.
4. Reads the predicted class value.
5. Rounds the value to an integer class.
6. Converts the class number to a gesture label.
7. Displays the class number and gesture label.
8. Changes the background color according to the class.
9. Shows debugging information.
10. Provides a reset key.

23. Important Output Sketch Variables

```
int inPort = 12000;
String oscAddress = "/wek/outputs";
```

These define how the output sketch receives Wekinator's predictions.

```
String class1Gesture = "Open Hand";
String class2Gesture = "Closed Fist";
```

These define the labels shown on screen.

```
int currentClass = 0;
String currentGesture = "Waiting for Wekinator...";
```

These store the current prediction.

24. Receiving OSC Output from Wekinator

The output sketch listens on port 12000:

```
oscP5 = new OscP5(this, inPort);
```

When a message arrives, Processing automatically calls:

```
void oscEvent(OscMessage msg)
```

The sketch first checks the OSC address:

```
if (!msg.checkAddrPattern(oscAddress)) {
  return;
}
```

This prevents the sketch from accidentally using the wrong OSC message.

Then it reads the first value:

```
lastRawValue = msg.get(0).floatValue();
```

For classification, Wekinator sends values like:

```
1.0
2.0
```

The sketch rounds the value:

```
currentClass = round(lastRawValue);
```

Then it converts the class number to a label:

```
currentGesture = classToGesture(currentClass);
```

25. Class-to-Gesture Mapping

The class mapping is handled by this function:

```
String classToGesture(int c) {
  switch(c) {
    case 1:
      return class1Gesture;
    case 2:
      return class2Gesture;
    default:
      return "Unknown Class";
  }
}
```

So:

```
1 → Open Hand
2 → Closed Fist
```

This makes the output easy to understand for the user.

26. Output Visualization

The output sketch shows:

1. The received class number.
2. The recognized gesture name.
3. A background color based on the class.
4. OSC debug information.
5. A small indicator dot that flashes when a message is received.

The background colors are:

```
Class 1 → red background
Class 2 → blue background
Waiting/unknown → dark background
```

This provides immediate visual feedback.

27. Correct Running Order

27.1. During Training

The output sketch is not required during recording, but it can be open.

Recommended training order:

```
1. Run GestureInput.pde.
2. Open Wekinator.
3. Start listening.
4. Record Class 1 examples.
5. Record Class 2 examples.
6. Set KNN to k = 3.
7. Train the model.
```

27.2. During Testing and Demonstration

All three components must run:

```
1. Run GestureInput.pde.
2. Open the trained Wekinator project.
3. Run GestureOutput.pde.
4. Click Run in Wekinator.
5. Show gestures to the camera.
```

The live testing pipeline is:

```
Hand gesture
    ↓
Input sketch sends features
    ↓
Wekinator predicts class
    ↓
Output sketch displays label
```

28. Important Practical Training Notes

28.1. The Hand Must Be Inside the Crop Box

This is the most important practical note.

Wekinator does not see the full webcam image.

It only receives the cropped and downsampled image.

Therefore:

```
If the green crop box contains the hand, Wekinator learns the gesture.
If the green crop box contains the face or background, Wekinator learns the wrong thing.
```

During both training and testing, the gesture should be clearly visible inside the green box.

28.2. Keep the Background Simple

A complicated background can affect pixel features.

A plain wall or simple background is better.

28.3. Keep Lighting Stable

Pixel values depend strongly on lighting.

If lighting changes between training and testing, classification may become worse.

28.4. Record Slight Variations

Do not record only one perfectly still pose.

During recording, move the hand slightly:

```
slightly left
slightly right
slightly closer
slightly farther
slightly rotated
```

This helps the classifier generalize.

28.5. Keep Classes Balanced

The number of examples for both classes should be similar.

The current project is balanced:

```
Class 1: 637
Class 2: 594
```

28.6. Retrain After Changes

If you change the classifier setting, add new examples, or delete examples, you must click **Train** again.

29. Discussion: Image Resolution

Exercise 2 asks us to think about how image resolution affects classification.

The possible grayscale feature sizes are:

Resolution	Number of Inputs	Description
200×200 grayscale	40,000	Very detailed but very high-dimensional
100×100 grayscale	10,000	Good detail, lower dimensionality
50×50 grayscale	2,500	Compact, fast, easier to train

29.1. 200×200 Resolution

Advantages:

- Keeps the most visual detail.
- Finger shapes and boundaries are more preserved.
- May help for subtle gestures.

Disadvantages:

- Creates 40,000 grayscale inputs.
- More difficult for Wekinator to process.
- Higher risk of overfitting.
- More sensitive to small position changes.

29.2. 100×100 Resolution

Advantages:

- Keeps a reasonable amount of detail.
- Less heavy than 200×200.
- Can be a good compromise.

Disadvantages:

- Still creates 10,000 inputs.
- Requires more training examples than 50×50.

29.3. 50×50 Resolution

Advantages:

- Only 2500 inputs.
- Faster and easier to train.
- Good for clearly different gestures.
- Lower computational cost.

Disadvantages:

- Loses fine detail.
- May not distinguish very similar gestures.

For this project, 50×50 grayscale is a good choice because open hand and closed fist have clearly different shapes.

30. Discussion: Grayscale vs RGB

Exercise 2 also asks us to compare grayscale and RGB representations.

30.1. Grayscale

Grayscale uses one value per pixel.

Feature count:

```
features = width × height
```

Advantages:

- Lower dimensionality.
- Easier to train.
- Less sensitive to color differences.
- Good for shape-based gestures.

Disadvantages:

- Removes color information.
- May be less useful if color is important.

30.2. RGB

RGB uses three values per pixel.

Feature count:

```
features = width × height × 3
```

Advantages:

- Preserves color information.
- May help if color is meaningful.

Disadvantages:

- Three times more inputs than grayscale.
- More sensitive to lighting and camera color balance.
- More difficult to train.

30.3. Current Choice

The current project uses grayscale because open hand vs closed fist is mainly a shape classification problem.

The important information is the silhouette and brightness pattern of the hand, not the color.

31. Feature Configuration Table

Exercise 2 allows all combinations of resolution and color mode.

Image Size	Grayscale Inputs	RGB Inputs
200×200	40,000	120,000
100×100	10,000	30,000
50×50	2,500	7,500

The current project uses:

```
50×50 grayscale = 2500 inputs
```

This is the safest first configuration because it is easier to train and test.

32. How the Current Project Meets the Exercise Requirements

Exercise Requirement	How This Project Fulfills It
Capture real-time video input	`GestureInput.pde` uses Processing `Capture`
Crop central image region	Input sketch crops a 200×200 center region
Generate downsampled image	Input sketch resizes crop to 50×50
Support grayscale representation	Input sketch uses brightness values when `useRGB = false`
Support RGB representation	Input sketch includes RGB mode when `useRGB = true`

Exercise Requirement	How This Project Fulfills It
Convert pixels into feature vectors	Input sketch loops through image pixels and creates OSC values
Send features to Wekinator	Input sketch sends `/wek/inputs` to port 6448
Train gesture classifier	Wekinator project uses 2-class classifier
Use machine learning	Wekinator uses KNN with k = 3
Define two gesture classes	Class 1 = Open Hand, Class 2 = Closed Fist
Receive prediction output	Output sketch listens to `/wek/outputs` on port 12000
Display text label	Output sketch displays `Open Hand` or `Closed Fist`
Document results and discussion	This README explains resolution, color mode, training, and performance considerations

33. Results

The system was configured and trained using:

```
50×50 grayscale input
2500 Wekinator inputs
2 gesture classes
KNN classifier with k = 3
1231 training examples
```

The two classes are:

```
Class 1 → Open Hand
Class 2 → Closed Fist
```

Expected behavior after training and running the model:

```
Open hand shown inside crop box
        ↓
Wekinator predicts Class 1
        ↓
Output sketch displays Open Hand
```

```
Closed fist shown inside crop box
        ↓
Wekinator predicts Class 2
        ↓
Output sketch displays Closed Fist
```

The project demonstrates the complete real-time interaction loop:

```
gesture → feature extraction → classification → visual feedback
```

34. Common Problems and Debugging

34.1. Wekinator Does Not Receive Inputs

Possible causes:

- Input sketch is not running.
- Wekinator is not listening.
- Wekinator input port is not 6448.
- Input OSC message is not `/wek/inputs`.
- Wekinator input count is not 2500.
- The input sketch sending is OFF.
- The webcam is not working.
- Processing Video library is missing.
- oscP5 library is missing.

Fix:

```
Run input sketch → Start listening in Wekinator → Check input count and port
```

34.2. Wekinator Receives Input but Training Is Bad

Possible causes:

- Hand is outside the green crop box.
- Crop contains mostly face or background.
- Too few examples.
- Class labels were mixed accidentally.
- Lighting changed.
- Background is too complex.
- Class examples are unbalanced.

Fix:

```
Delete bad examples or create a new project.
Record again with the hand clearly inside the crop box.
Train again.
```

34.3. Output Sketch Does Not Change

Possible causes:

- Wekinator is not in Run mode.
- Output sketch is not running.
- Wekinator output port is not 12000.
- Wekinator output message is not `/wek/outputs`.
- Wekinator output type is not classification.
- Number of outputs is not 1.

Fix:

```
Run output sketch → Check port 12000 → Click Run in Wekinator
```

34.4. Output Shows Wrong Gesture

Possible causes:

- The model was trained with bad examples.
- The hand position during testing is different from training.
- Lighting is different.
- The crop contains background instead of hand.
- KNN was not retrained after changing settings.

Fix:

```
Record better examples, retrain, and test again.
```

35. Important Notes About the Green Crop Box

The green crop box is not just a visual decoration.

It represents the image region that becomes the feature vector.

Wekinator does not know what is outside the crop.

If the hand is outside the green box, Wekinator cannot classify the gesture properly.

Correct training example:

```
Green crop box contains hand gesture clearly.
```

Incorrect training example:

```
Green crop box contains face/background while the hand is outside.
```

This is one of the most important reasons why training may fail.

36. What Was Learned

36.1. Real-Time Video Processing

The project shows how Processing can capture live webcam video and process it frame by frame.

36.2. Image Preprocessing

The project demonstrates basic computer vision preprocessing:

```
crop → resize → grayscale conversion → pixel vector
```

36.3. Feature Extraction

The project shows that raw images must be transformed into numerical feature vectors before machine learning can be applied.

36.4. Dimensionality

The project demonstrates that feature dimensionality strongly affects machine learning performance.

A larger image gives more detail but creates more inputs.

A smaller image is easier to train but loses detail.

36.5. OSC Communication

The project uses OSC to connect independent programs in real time.

36.6. Supervised Learning

The project demonstrates supervised learning because the user records labeled examples for each class.

36.7. KNN Classification

The project demonstrates how KNN can classify new inputs by comparing them with stored training examples.

36.8. Interactive Machine Learning

The complete system is interactive because the user can train examples, test gestures, and receive real-time feedback.

37. Possible Improvements

37.1. Movable Crop Box

The current crop box is centered. A useful improvement would be allowing the user to move the crop box with the keyboard or mouse.

This would make it easier to place the crop exactly over the hand.

37.2. Larger Feature Size

Testing 100×100 grayscale could preserve more detail than 50×50 while still being manageable.

37.3. Background Removal

A more advanced version could separate the hand from the background before sending features.

37.4. More Gesture Classes

The system could be extended to more than two classes, such as:

```
Open hand
Closed fist
Thumbs up
Two fingers
Pointing
```

This would require updating Wekinator and the output sketch.

37.5. More Robust Training

The classifier could be improved by recording examples under different lighting, hand positions, and distances.

38. Final Project Summary

This project implements a complete real-time gesture classification system.

The input sketch captures webcam images, crops a 200×200 central region, downsamples it to 50×50, converts it into grayscale pixel values, and sends 2500 features to Wekinator using OSC.

Wekinator receives these features on port 6448 using the `/wek/inputs` message. It is configured as a two-class classifier with one output and uses KNN with `k = 3`. The two classes are Open Hand and Closed Fist. The project contains 1231 training examples, balanced between the two classes.

The output sketch listens on port 12000 for `/wek/outputs`. It receives the predicted class from Wekinator, converts the class number into a gesture label, and displays the recognized gesture on the screen.

The complete pipeline is:

```
Webcam gesture
    ↓
Processing input sketch
    ↓
Image crop and downsampling
    ↓
2500 grayscale pixel features
    ↓
OSC /wek/inputs
    ↓
Wekinator KNN classifier
    ↓
OSC /wek/outputs
    ↓
Processing output sketch
    ↓
Gesture label display
```

The project satisfies the main requirements of Exercise 2 and demonstrates key concepts in real-time video processing, feature extraction, OSC communication, supervised machine learning, KNN classification, and interactive visual feedback.

39. Final Conclusion

Exercise 2 was successfully implemented as a real-time gesture classification system using Processing and Wekinator.

The project uses the webcam as input, extracts visual pixel features from a cropped and downsampled image, trains a KNN classifier in Wekinator, and displays the predicted gesture label in a second Processing sketch.

The final tested configuration is:

```
Input: webcam
Crop: 200×200
Feature image: 50×50
Color mode: grayscale
Wekinator inputs: 2500
Classifier: KNN
k: 3
Classes: 2
Class 1: Open Hand
Class 2: Closed Fist
Output: gesture label display
```

This configuration is a good practical choice because it balances simplicity, speed, and enough visual information to distinguish clearly different gestures.

The most important lesson from this exercise is that machine learning performance depends strongly on feature quality. Wekinator can only learn from the data it receives. Therefore, correct cropping, stable lighting, balanced examples, and consistent training conditions are essential for reliable gesture classification.